

🎯 本章目标

理解 C++ 的编译模型:翻译单元、头文件文本包含与链接;掌握 include guard 的两种写法与 #pragma once 的取舍;学会头文件与源文件的分离纪律、前置声明与 Pimpl;看懂 C++20 模块(modules)如何替代头文件,并能在新项目中动手使用 import/export。

📦 前置知识

熟悉函数、类、模板的声明与定义(第4、6、10章);了解链接错误(第1章);本章代码 C++17 可编译,C++20 模块部分需要 C++20 编译器(GCC 11+、Clang 16+、MSVC 2019 16.11+)。

§13.1 编译模型	§13.2 头文件纪律	§13.3 分离编译
§13.4 ODR 与多定义	§13.5 包含成本	§13.6 C++20 模块

§13.1 编译模型:#include 是文本粘贴

C++ 从 C 继承了头文件与源文件的二分法。一个 .cpp 文件连同它包含的所有头文件,展开成一个翻译单元(translation unit)。`#include` 的语义朴素得惊人:把另一个文件的内容原样粘进来。也就是说,"头文件"在 C++ 标准里并不存在,只有预处理后的文本;你看到的 `#include <iostream>` 在编译真正开始前,已经被替换成了上千行标准库代码。理解这一点,才能理解后面所有关于头文件的规则。

编译的产物是目标文件(object file),里面记录每个函数和变量的定义,以及"我用到但没定义"的符号引用。链接器负责把多个目标文件与库拼在一起:找到每个符号唯一的定义,填上引用。于是出现两类型错误:声明了却没人定义(链接错误 undefined reference),和同一个实体定义了多份(链接错误 multiple definition)。模块系统要解决的,就是这套文本粘贴模型带来的重复、泄漏与低效。

设计上要记住一个原则:头文件里放声明与内联/模板定义,源文件里放普通定义。声明是"对外承诺",可以重复出现多次而不冲突;定义是"唯一事实",在程序里只能有一份。

C 的写法	C++ 的写法	说明
<code>#include</code> 文本粘贴,无其他选择	头文件之外,C++20 有模块可选	模块提供语义级导入,替代文本展开
头文件内容对包含方全部可见	模块里未 <code>export</code> 的符号不可见	C++20 起可见性由语言保证
每次包含都重复解析文本	模块预编译一次,import 零重复开销	大型项目编译时间收益显著

设计上要记住一个原则:头文件里放声明与内联/模板定义,源文件里放普通定义。声明是"对外承诺",可以重复出现多次而不冲突;定义是"唯一事实",在程序里只能有一份。谁违反了这个分工,谁就制造链接错误或编译膨胀。接下来的小节把这条原则展开成可操作的纪律。

§13.2 头文件纪律:guard 与自包含

头文件被多个源文件包含,甚至被同一个源文件包含两次,是家常便饭。没有防护,同一个类会被定义两遍,编译直接失败。`include guard` 就是在头文件首尾设置的"开关":第一次进入时定义宏并贴内容,以后再来,宏已存在,整段跳过。经典写法用宏名(工程名+路径+随机串),另一种写法 `#pragma once` 由编译器直接识别"这个文件只处理一次",更简洁且没有宏名冲突问题,代价是不被旧标准保证、某些诡异场景(同一文件通过不同路径包含、符号链接)可能失效。

清单 13-1 include guard 两种写法

```
// 写法 A:经典宏 guard(一切编译器可用)
#ifndef CPP_TUTOR_MATH_UTILS_H_
#define CPP_TUTOR_MATH_UTILS_H_

int add(int a, int b);
double avg(const double* xs, int n);

#endif // CPP_TUTOR_MATH_UTILS_H_

// 写法 B:#pragma once(GCC/Clang/MSVC 都支持)
#pragma once

int mul(int a, int b);
```

现代工程几乎全部用 #pragma once:不用操心宏名唯一、少两行样板、可读性好。唯一推荐经典宏的场景是要兼容远古编译器,或者项目规范强制。但注意:#pragma once 是按"文件路径"去重的,如果同一内容被复制成两份不同文件,它拦不住;宏 guard 至少能通过同一个宏名拦住。工程上不必为此焦虑,二选一,保持一致即可。

更重要的纪律是"头文件自包含":任何头文件,单独 #include 它就应该能编译通过。检查方法:把每个头文件放进一个空的 .cpp 里包含并编译,不通过就修。自包含的收益是使用方不必记住"要先包含它的依赖",排序自由,依赖不泄漏。与自包含配套的是每个头文件只包含自己真正需要的东西,能用前置声明就不用 #include(详见 13.5),能不放定义就不放定义。

§13.3 分离纪律:声明与定义各就各位

头文件与源文件的经典分工:接口放 .h(类的成员声明、自由函数声明、常量、内联函数、模板定义),实现放 .cpp(成员函数定义、自由函数定义、静态变量)。使用者只看头文件就能调用,实现改动只重编译库本身,不牵连所有调用方——这是分离编译最大的工程价值:编译时间与依赖面都被压缩。

清单 13-2 头文件与源文件分离

```

// ---- math_utils.h ----
#pragma once
#include <vector>

// 类:成员声明在头文件
class Stats {
public:
    void add(double v);           // 声明
    double mean() const;         // 声明
private:
    std::vector<double> values_;
};

// 自由函数声明
double variance(const std::vector<double>& xs);

// 内联函数定义可以放头文件
inline double square(double x) { return x * x; }

// ---- math_utils.cpp ----
#include "math_utils.h"
#include <cmath>
#include <numeric>

void Stats::add(double v) {      // 定义:类外写,带类名限定
    values_.push_back(v);
}

double Stats::mean() const {
    return values_.empty() ? 0.0
        : std::accumulate(values_.begin(), values_.end(), 0.0)
            / static_cast<double>(values_.size());
}

double variance(const std::vector<double>& xs) {
    if (xs.empty()) return 0.0;
    Stats s;
    for (double x : xs) s.add(x);
    double m = s.mean(), sum = 0.0;
    for (double x : xs) sum += (x - m) * (x - m);
    return sum / static_cast<double>(xs.size());
}

```

哪些东西必须放进头文件:类定义(成员声明+内联实现)、函数模板与类模板的完整定义、inline 函数、constexpr 函数、inline 变量、以及 enum class。它们的共同点是编译器在调用点必须看到完整定义才能生成代码,或语言允许它们有多份相同定义(内联与模板的"多份相同可接受"规则见 13.4)。普通函数、全局变量的定义则一律放 .cpp,只把声明留在头文件。

还有两个细分技巧。其一,头文件里尽量少放 #include,能用前置声明就用;这样头文件的依赖变薄,包含它的文件不会被连锁拖累。其二,类的数据成员不变的前提下,头文件应该"常年稳定":公共头文件一改,全项目重编译,所以把实现细节藏进 .cpp 是贵族级素养——最极致的形式是 Pimpl(pointer to implementation)。

清单 13-3 Pimpl:把实现藏进源文件

```
// ---- widget.h:只暴露稳定接口 ----
#pragma once
#include <memory>

class Widget {
public:
    Widget(); // 析构等特殊成员必须在 cpp 定义
    ~Widget();
    Widget(Widget&&) noexcept;
    Widget& operator=(Widget&&) noexcept;
    Widget(const Widget&) = delete; // 复制代价大,直接禁掉
    Widget& operator=(const Widget&) = delete;

    void setText(const char* text);
    const char* text() const;
private:
    struct Impl; // 前向声明,不泄露任何成员
    std::unique_ptr<Impl> p_; // 智能指针持有不完整类型
};

// ---- widget.cpp ----
#include "widget.h"
#include <string>

struct Widget::Impl { // 真正的成员都在这里
    std::string text = "空";
    int version = 1;
};

Widget::Widget() : p_(std::make_unique<Impl>()) {}
Widget::~~Widget() = default; // 这里才需要完整类型
Widget::Widget(Widget&&) noexcept = default;
Widget& Widget::operator=(Widget&&) noexcept = default;

void Widget::setText(const char* t) { p_->text = t ? t : ""; }
const char* Widget::text() const { return p_->text.c_str(); }
```

C 的写法	C++ 的写法	说明
.c 文件里 extern 声明共享变量	头文件 extern 声明 + .cpp 定义, 或 inline 变量	C++17 起头文件可安全放共享变量
泛型靠宏与 void*	模板定义必须放头文件(或模块)	模板是 C++ 头文件文化的最大来源
头文件里的宏污染无法根除	constexpr、inline、命名空间替代宏	头文件里的 #define 是最难查的 bug 源之一

Pimpl 的收益:widget.h 里没有任何标准库头文件依赖,调用方编译飞快;Impl 里加成员、加头文件,使用方零感知。代价是每次访问成员多一层指针解引用、对象分配堆内存、移动语义要自己维护。它适合"被广泛包含且长期稳定"的库类型,不适合性能热点上的小对象。C++20 模块出现后,Pimpl 的地位有所松动,但存量代码里它仍是主力武器。

§13.4 ODR:为什么"多定义"有时合法

链接器发现同一个符号有两份定义时报错,但内联函数、模板、类定义明明会被每个包含它们的翻译单元各生成一份——为什么它们不冲突?这靠的是 ODR(单一定义规则)的例外条款:如果这些实体在多处定义,只要每处定义"完全相同"(token 级一致),就视为同一实体,链接器取一份即可。这是头文件模型能成立的理论基石:头文件内容在所有包含点相同,于是类与模板的重复定义被豁免。

ODR 的两个实操教训。第一,内联函数与模板的"完全相同"要求很脆:不同翻译单元里宏定义不同,展开出不同的函数体,就违反了 ODR,行为未定义,而且经常编译期不报错、运行期诡异。排查办法:禁止在头文件里依赖条件编译改变接口含义。第二,普通函数与全局变量没有豁免:定义放 .cpp,头文件里只放声明,否则换来的就是 multiple definition 链接错误。

清单 13-4 违反 ODR 的典型现场

```
// ---- config.h(错误示范!)----
#pragma once
#include <string>

// 问题 1:非内联函数定义放头文件,两个 cpp 各展开一份
// std::string version() { return "1.0"; } // 链接错误

// 问题 2:全局变量定义放头文件
// int g_debugLevel = 2; // 链接错误

// 正确做法:声明放头文件
std::string version();
extern int g_debugLevel;

// ---- config.cpp ----
#include "config.h"
std::string version() { return "1.0"; }
int g_debugLevel = 2;

// ---- main.cpp ----
#include "config.h"
#include <iostream>
int main() {
    g_debugLevel = 3; // 用 extern 声明访问全局变量
    std::cout << version() << "\n";
    return 0;
}
```

C++17 给"头文件里想放全局变量"开了一个口子:inline 变量。inline 变量的定义可以出现在多个翻译单元且取同一份存储,常用于头文件里的配置常量、单例指针等场景。它的行为与函数模板的"多份相同"一致,是安全的。但别滥用:能用函数调用封装的状态,优先用函数(局部静态变量),可读性更好,初始化顺序问题也更少。

§13.5 包含成本与循环依赖

头文件模型最大的隐形成本是编译时间。每个 #include 都是文本粘贴:一个头文件的 #include 会拉进它的传递依赖,一行头文件改动引发全项目级联重编译。百万行 C++ 项目的编译时间以十分钟计,头文件贡献了绝大部分。缓解手段

按性价比排序:前置声明代替 #include;头文件瘦身(少包含、少放实现);Pimpl;统一编译头(precompiled header)把高频稳定头一次编译缓存;再就是迁移到 C++20 模块。

前置声明(forward declaration)是只声明"存在这么个类",不展开它的定义:class Logger;。代价是不能定义该类型的对象、不能调用成员、不能 delete 指针——只能声明指针/引用、用作函数参数与返回类型。对"互相引用"的类,前置声明是唯一的解药:A 有 B 的指针、B 有 A 的指针,两个头文件互相 #include 就是死循环,guard 虽然能阻止编译崩溃,却让其中一方在包含点看不到完整类型而报错。正确结构是:各自只前置声明对方,把需要完整类型的实现放进 .cpp,在 .cpp 里 #include 对方头文件。

清单 13-5 前置声明解开循环依赖

```
// ---- a.h ----
#pragma once
class B;                // 前置声明:只知道存在 B

class A {
public:
    void greet(const B& b) const;    // 引用参数,不完整类型就够
    B* partner() const { return partner_; }
private:
    B* partner_ = nullptr;    // 指针,不完整类型就够
};

// ---- b.h ----
#pragma once
class A;

class B {
public:
    void greet(const A& a) const;
};

// ---- a.cpp ----
#include "a.h"
#include "b.h"          // 实现里才需要完整类型
#include <iostream>

void A::greet(const B& b) const {
    std::cout << "A 对 B 说:你好\n";
    b.greet(*this);
}

// ---- b.cpp ----
#include "b.h"
#include "a.h"
#include <iostream>

void B::greet(const A& a) const {
    std::cout << "B 对 A 说:你好\n";
    (void)a;
}
```

包含顺序也是一门纪律:头文件之间相互包含的次序不稳定是毒药,工程标准做法是"每个头文件自包含 + 包含者先包含自己的头文件再包含标准库",把顺序敏感性连根拔掉。头文件里的宏污染要警惕:宏是全局的文本替换,一个头文件里

#define 的宏可能改写到处使用方代码里的同名标识符。这也是 C++ 讨厌宏、头文件尽量少用宏的又一个理由。

§13.6 C++20 模块:告别文本粘贴

模块(modules)是 C++20 对头文件模型的根本替代。模块文件(.cppm 等)内部声明 import 进来的依赖、export 出去的接口;使用方写 import math_utils; 即可,编译器把模块编译成带语义的二进制(带 GCC/Clang/MSVC 各自的实现格式),使用方直接读取语义,不再展开文本。好处是本质性的:接口与实现分离由语言保证(没 export 的符号对使用方不可见,宏更是无从泄漏);编译时间大幅下降(模块只编译一次,不随使用方数量翻倍);编译依赖顺序友好(无循环包含问题)。

清单 13-6 C++20 模块最小示例

```
// ---- math_utils.cppm(模块单元)----
export module math_utils;           // 导出模块名
import <vector>                      // 导入标准库头(模块化兼容)

export class Stats {                 // export 的才对外可见
public:
    void add(double v) { values_.push_back(v); }
    double mean() const;
private:
    std::vector<double> values_;
};

export double variance(const std::vector<double>& xs);

// 未 export 的实现细节:对使用方不可见
double Stats::mean() const {
    double sum = 0;
    for (double v : values_) sum += v;
    return values_.empty() ? 0.0 : sum / values_.size();
}

double variance(const std::vector<double>& xs) {
    Stats s;
    for (double v : xs) s.add(v);
    return s.mean();
}

// ---- main.cpp ----
import math_utils;                  // 使用方:一行 import
import <iostream>

int main() {
    Stats s;
    s.add(1); s.add(2); s.add(6);
    std::cout << "平均: " << s.mean() << "\n";
    return 0;
}

// 编译(以 GCC 为例,先编模块再编主程序):
// g++ -std=c++20 -fmodules-ts math_utils.cppm main.cpp -o app
```

模块与头文件的关键差异要记住:一是可见性,模块里没 export 的一切(实现函数、辅助类型、宏)对使用方完全不可见,头文件模型里这些全在公共视界;二是包含成本,import 是读取预编译好的语义信息,与文本大小无关,重复 import 也不

重复开销;三是构建顺序,模块必须先于使用方编译,构建系统需要了解模块依赖图,而头文件只需要文件存在。四是兼容性,import <iostream> 这类"模块化头"是标准库的福利,第三方老库只能 import "xxx.h" 用头文件模式过渡。

迁移策略:新项目、新库优先用模块;存量大型项目按依赖自底向上把最稳定、最被广泛包含的底层库先迁(收益最大),边界处用 import "old.h" 过渡;编译命令与构建脚本要跟上(CMake 的 C++20 模块支持在 3.28+ 已可用)。模板在模块里不再需要"定义必须在调用处可见"——模块内模板定义会被编进模块二进制,使用方 import 即可实例化,这解决了头文件模型最大的痛点。工具链对模块的报错信息、IDE 支持尚在追赶,这是 2020 年代 C++ 工程的现实。

C 的写法	C++ 的写法	说明
#ifndef 宏 guard 是唯一选择	#pragma once(或 C++20 模块)	C 无标准替代,只能宏 guard;C++ 有编译器级与语言级方案
头文件里无法声明"别的东西不可见"	模块里未 export 的符号天然隐藏	C 的 #include 是全文可见,模块有可见性边界
包含顺序与宏污染全凭自觉	import 无顺序问题、无宏泄漏	模块从机制上消灭两类头文件顽疾

⚠ 易错点 1:guard 宏名重复,头文件被吞

两个毫不相干的头文件若写了同一个 guard 宏名(常见于复制粘贴),谁先被包含,谁就永久屏蔽另一个:第二个头文件的内容整个被跳过,缺声明,编译报"未定义"或类未声明的错,且报错位置远离源头。guard 宏名要带上项目与路径特征(如 CPP_TUTOR_MATH_UTILS_H),或干脆用 #pragma once。

⚠ 易错点 2:头文件放非内联定义

把函数定义、全局变量定义写进头文件,每个包含它的 .cpp 都会生成一份,链接时报 multiple definition。新手最常见:定义写在头文件里恰好单文件程序能跑,一旦多文件立刻炸。守则:普通函数与全局变量只声明进头文件;定义交给 .cpp。类内定义的成员函数、模板、inline 函数才允许(且必须)在头文件。

⚠ 易错点 3:模板定义放在 .cpp,链接失败

模板实例化发生在使用点:调用方编译时若看不到模板定义,只看到声明,会生成"未定义引用"链接错误。把模板定义塞进 .cpp 只编译了 .cpp 里用到的实例,使用方仍然找不到。模板要么整体放头文件,要么在 .cpp 末尾显式实例化(export template),要么用模块。头文件里写模板是 C++ 的常态,不是妥协。

快速参考:头文件与模块速查表

场景	做法
头文件防重复包含	#pragma once(兼容旧编译器用宏 guard)
头文件自包含检查	每个头文件单独包含进空 cpp 编译
类成员声明	头文件(定义在 .cpp 类外写)
类内实现的小函数	头文件类内定义,隐式内联
模板定义	头文件(或模块)
普通函数定义	.cpp,头文件只留声明
全局变量定义	.cpp + extern 声明;或头文件 inline 变量
互相引用的类	各自前置声明,实现放 .cpp
隐藏实现细节	Pimpl(unique_ptr + 不完整类型)
减少包含成本	前置声明、头文件瘦身、PCH、模块
C++20 模块导出接口	export module / export 声明
模块使用	import 模块名;编译时先编模块单元
ODR 豁免实体	inline 函数/变量、模板、类定义(内容须相同)

自测题

Q 1. #include 在预处理阶段做了什么?为什么说头文件"文本粘贴"模型决定了后面所有规则?

✓ 答案

把被包含文件的内容原样展开进当前翻译单元。编译从展开后的文本开始,因此头文件内容在所有包含点展开后必须一致(否则违反 ODR),头文件内容全部对包含方可见(因此要自己管 guard、管可见性),每次包含都要重新解析文本(因此有编译成本)。

Q 2. 一个头文件既被 a.cpp 又被 b.cpp 包含,里面有普通函数定义,链接会发生什么?为什么模板定义却没事?

✓ 答案

普通函数定义没有 ODR 豁免,两份不同翻译单元的定义产生 multiple definition 链接错误。模板定义允许"多份相同定义"(内容 token 级一致即视为同一实体),由链接器合并,所以模板定义放头文件是安全的。

Q 3. 前置声明 class B; 之后,哪些操作合法,哪些非法?

✓ 答案

合法:声明 B 的指针/引用、作为函数声明参数与返回类型。非法:定义 B 的对象(B 的大小未知)、访问 B 的成员、对 B* 做 delete(析构未知)、继承 B、用 sizeof(B)。需要这些操作的代码必须移进 .cpp 并包含完整头文件。

Q 4. Pimpl 技法把什么放进了 .cpp?它的主要收益与代价是什么?

✅ 答案

把类的全部数据成员与实现细节放进嵌套类 Impl,头文件只剩前向声明与 unique_ptr 成员。收益:头文件零依赖,调用方编译快,实现改动不影响使用方重编译。代价:每次访问多一层指针间接、对象在堆上、移动/析构特殊成员要在 .cpp 里定义,小对象性能敏感时慎用。

Q 5. C++20 模块相比头文件,解决了哪三类本质问题?

✅ 答案

可见性:未 export 的符号与宏对使用方不可见;成本:模块预编译一次、import 按语义读取,重复包含无开销,编译时间大幅下降;健壮性:无包含顺序问题、无循环依赖、无宏污染,模板定义也随模块分发。

实验:一个头文件组织的日志库

本实验写一个日志库,把本章全部知识点落地:Log.h 用 #pragma once、自包含、只放声明与 inline 函数;Log.cpp 放实现;main.cpp 使用它;再演示模板声明与定义分离、inline 变量、以及 Pimpl 的变体(日志实现细节藏在 .cpp 的内部匿名命名空间)。运行后应输出 INFO/ERROR 各一行到控制台与 log.txt。

实验:Log.h 头文件

```
// ---- Log.h ----
#pragma once                // 覆盖:include guard
#include <string>             // 自包含:自己包含自己需要的头

namespace tut {              // 命名空间收纳,避免污染全局

// inline 变量:多翻译单元共享一份存储
inline int g_verbose = 1;    // 覆盖:inline 变量

enum class Level { Info, Warn, Error };

// 模板:必须整体放头文件
template <typename T>
inline std::string fmt(const T& v) {
    return std::to_string(v); // 覆盖:模板定义在头文件
}

class Logger {               // 声明在头文件
public:
    Logger();                // Pimpl 风格:构造在 cpp 完成
    ~Logger();
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;
    void log(Level lv, const std::string& msg);
    void setFile(const std::string& path);
private:
    struct Impl;              // 覆盖:前置声明
    Impl* p_;                 // 实现细节在 cpp
};

} // namespace tut
```

实验:Log.cpp 实现

```
// ---- Log.cpp ----
#include "Log.h"
#include <fstream>
#include <iostream>
#include <mutex>

namespace tut {

struct Logger::Impl {          // 覆盖:定义藏在 cpp,不泄露
    std::ofstream file;
    std::mutex mtx;
};

Logger::Logger() : p_(new Impl()) {}    // 覆盖:构造分配实现
Logger::~Logger() { delete p_; }       // 覆盖:析构释放

void Logger::setFile(const std::string& path) {
    p_>file.open(path, std::ios::app);
}

void Logger::log(Level lv, const std::string& msg) {
    const char* tag = (lv == Level::Info) ? "INFO"
                    : (lv == Level::Warn) ? "WARN" : "ERROR";
    std::lock_guard<std::mutex> lock(p_>mtx); // 覆盖:并发安全
    std::string line = std::string(tag) + ": " + msg;
    std::cout << line << "\n";
    if (p_>file) p_>file << line << "\n";
}

} // namespace tut
```

实验:main.cpp 使用

```
// ---- main.cpp ----
#include "Log.h"
#include <string>

int main() {
    tut::Logger logger;                // 覆盖:分离编译的使用方
    logger.setFile("log.txt");

    logger.log(tut::Level::Info, "启动, 版本=" + tut::fmt(3));
    tut::g_verbose = 0;                // 覆盖:inline 变量跨单元共享
    logger.log(tut::Level::Error, "磁盘空间不足");

    // 模板在头文件,使用方实例化 std::string 版本
    std::string s = tut::fmt(std::string("字符串"));
    (void)s;
    return 0;
}

// 编译(Windows 命令行 / Linux 同理):
// g++ Log.cpp main.cpp -std=c++17 -o logger_app
// 运行后控制台与 log.txt 各出现 INFO 与 ERROR 两行
```

工程实践 1:头文件也是接口,评审它

头文件是 C++ 的公共接口层,评审代码时先看头文件:依赖是否最少、是否自包含、是否只暴露必要的 API、注释是否说明前置条件与线程安全。接口稳定的库,头文件一年不动,团队编译速度就赢了一半。

工程实践 2:增量迁移到模块

别等"全项目迁移"的完美时机:把最稳定、被包含最广的底层库先迁成模块,收益立刻显现;旧代码 import "老.h" 照常工作。同时把构建脚本的模块支持打开(CMake 3.28+、Bazel 6+),新文件一律用模块写,两年后回头看,头文件只剩一小撮。

本章总结

头文件是 C 时代的文本粘贴机制,理解翻译单元与链接才能写出正确的多文件工程:guard 防重复、自包含保稳定、声明与定义分居头/源、模板与 inline 靠 ODR 豁免留在头文件、前置声明与 Pimpl 控制依赖与编译成本。C++20 模块用 import/export 取代 #include,从语言层面解决可见性、编译时间与宏污染,是头文件体系的接班人。无论是维护老工程还是开新项目,模块系统的知识都直接决定编译体验与接口卫生。